

a new suite of drivers or design another query language, we chose to leverage the work that had already gone into drivers, the language and the applications that use them. Scylla is Cassandra-compatible in the wire protocol, JMX monitoring, the underlying file format and even the configuration file.

As a result of this fundamental decision, Cassandra users migrating to Scylla are spared from having to learn an entirely new data model. Their applications will work just as before and they can continue to leverage their existing ecosystems.

### Design decision 3: All things asynchronous

Modern hardware works in an asynchronous manner. Equipped with multi-core processors, these machines are capable of performing hundreds of thousands of I/O operations per second. However, in order to harness the full power of this hardware, you need asynchronous software as well.

In systems programming, there are a lot of things going on at the same time. For instance, there are multiple queries running at the same time on different cores. These queries could be waiting for data from other cores or from other nodes on the network. They could be waiting for data to arrive from the disk or they could be waiting for the cloud to provide more data.

By making everything asynchronous, we've created a uniform way to address the problem of waiting for something to arrive. While building a framework that handles everything asynchronously meant more upfront work for us, once it was done, we no longer had to address the problems that could arise when increasing concurrency. With this approach, the number of concurrent queries is limited not by the framework itself, but by the extent of the system's resources.

It's worth noting that Scylla uses Seastar, our async engine. Here, you can read how Scylla accesses the disk with async and direct DMA using the Linux API. The Redis project (Parallel Redis) uses the Seastar engine for parallelism. Another project which uses Seastar is SMF, an ultra-fast RPC designed by Alexander Gallego of Akamai.

### Design decision #4: Shard per core

With the normal programming model of having a large number of threads, you're guaranteed to fight scalability problems every step of the way. A threaded programming model helps the program to focus on the problem and use locks to exclusively access the data. However, with Moore's Law being stretched to more cores instead of higher frequency, the core count grows and thus the contention. When you acquire spin locks, all of the other threads cannot access the data and are unable to proceed. Even granular locks don't help since locking the bus to acquire a spin lock costs 20 per cent of the CPU's time even when the lock is not contended.

The complexity of the traditional threaded model doesn't end with locking. With more than a single socket, memory

accesses may spread across sockets, but access to a memory bank that belongs to a neighbouring socket costs 2x that of a local socket. It's very hard to code memory with NUMA oriented allocations.

Lastly, there are the I/O devices. Modern NICs and drives have multi-queue modes, where every CPU can drive I/O. The standard model usually does not have enough handlers or will cause a lot of context switches.

Scylla solves all these scalability issues by using a shared-nothing architecture. The data set is sharded into smaller ranges called shards. This is the same process as sharding a whole data set from cluster to nodes. It is completely hidden from the user.

Scylla runs multiple shards per server node. The number of shards is usually the same as the number of CPU cores or, more accurately, hyper threads. Each shard is an independent unit and fully owns its data set. A shard has a thread of execution, a single OS-level thread that runs on that core and a subset of the RAM. The thread and the RAM are pinned in a NUMA friendly way. Each shard issues its own I/O, either to the disk or to the NIC, directly.

Shards communicate using shared-memory queues, one for each pair of source:target shards. Requests that need to retrieve data from several shards are first parsed by the receiving shard, then distributed to the target shard in a map-reduce fashion using Seastar's queues.

Each shard is executed by a single thread. However, the shard has different tasks to execute, such as network exchange, disk I/O, compaction, and reads and writes. That's why we developed a task scheduler that schedules tiny lambda functions (continuations). It has low overhead in

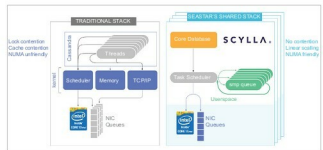


Figure 1: Traditional vs Seastar's shared stack

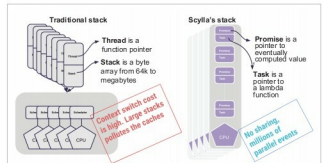


Figure 2: Scylla's thread-per-core architecture